

Evaluation and Replication Study of Data Science

Methods for Phishing Detection

Article: Phishing Detection Using Machine Learning Techniques

Authors: Vahid Shahrivari, Mohammad Mahdi Darabi, and Mohammad Izadi

Course: Data Science in Cybersecurity

Lecturer: Dr. Uri Itai

Submitted by: Loay Arkab

ID: 324893684

Due: July 19th 2026

1. Summary of the Source

• The Problem Being Addressed

The selected paper addresses the critical security challenge of detecting phishing websites. Phishing is a form of social engineering cyberattack where malicious actors construct fraudulent websites that visually mimic legitimate, trusted entities (such as banks, social media platforms, or e-commerce sites). The core problem is that these deceptive sites are designed to trick unsuspecting human users into revealing highly sensitive and confidential information, including login credentials, personal identification numbers, and credit card details. From a data science perspective, the paper treats this issue as a supervised binary classification problem, where a system must accurately analyse a set of extracted web features and label a given URL or domain as either "legitimate" or "phishing".

• Why the Problem is Important

Phishing remains one of the most common and damaging vectors for cybercrime worldwide. Despite the advancement of traditional security infrastructures, human users continue to be the weakest link in the security chain, making social engineering highly effective. This problem is important for several reasons:

Financial and Reputational Damages: Successful phishing attacks lead to massive financial losses for individuals through unauthorized transactions and identity theft. For enterprises, a phishing breach can compromise internal networks, leaking proprietary data, causing severe regulatory penalties, and destroying customer trust.

Failure of Traditional Approaches: Historically, phishing detection relied heavily on blacklisting and whitelisting. While effective for known threats, these methods fail completely against zero-day phishing attacks, where a malicious URL is newly generated and remains unlisted. Attackers frequently use automated scripts to create short lived domains that bypass static lists within minutes.

Scale and Velocity: The volume of phishing sites created daily is staggering. Security operation centers (SOCs) cannot manually inspect every suspicious link. Therefore,

developing robust, automated, and intelligent systems capable of inspecting web indicators in real time is crucial to protecting internet users before they fall victim to fraud.

• The Proposed Solution

To counter the limitations of static blacklists, the authors propose a machine-learning based framework for automated phishing detection. The core idea is to shift the security paradigm from analysing the reputation of a URL (which can easily be altered) to evaluating the intrinsic behavioural, structural, and semantic characteristics of the webpage itself. The proposed solution involves extracting multi dimensional features from a website—including URL patterns, HTML structure, and external domain authority signals—and passing them into a trained suite of supervised learning models. By training these models on historical instances of verified phishing and legitimate sites, the system learns the underlying statistical boundaries that separate authentic web behavior from deceptive tactics, allowing it to predict the nature of unseen URLs dynamically.

• The Dataset Used

The study utilizes the widely recognized **Phishing Websites Dataset** from the UCI Machine Learning Repository, a standard benchmark in cybersecurity data science research. The dataset consists of **11,055 instances** in total. The target variable is binary, representing a well defined classification space:

Legitimate Websites: 6,157 instances

Phishing Websites: 4,898 instances

The dataset contains **30 distinct features** extracted from the pages, which are structurally categorized into four main categories:

Address Bar Based Features: Characteristics of the URL itself, such as the length of the URL, the presence of the @ symbol, IP address utilization instead of a domain name, and sub domain redirection counts.

Abnormal Features: Irregularities within the connection parameters, such as the use of request URLs pointing to external domains or an altered address bar.

HTML and JavaScript Features: Client-side source code traits, including the redirection of scripts, the disablement of right click functionalities, or form submissions (<form>) pointing to blank pages or external handlers.

Domain Based Features: Statistical reputations of the web entity, including the age of the domain, DNS records, and website traffic ranking metrics.

Crucially, the raw features are already preprocessed into discrete categorical values, typically represented as integers: 1 (Legitimate), 0 (Suspicious), or -1 (Phishing).

• The Model or Methodology Employed

The authors implement a comparative empirical methodology to evaluate how various mathematical architectures handle the extracted cybersecurity features. The pipeline follows a standard machine learning workflow: loading the dataset, performing an 80/20 train-test split, and training multiple classifiers. The core models investigated in the source paper include:

Logistic Regression: Used as a baseline probabilistic linear model to evaluate structural separability.

Decision Trees (C4.5 / CART variations): Employed to capture non-linear, rule-based splits within the features.

Random Forest: An ensemble method utilizing bagging to minimize variance and combat overfitting across the 30 features.

Support Vector Machines (SVM): Applied with different kernel functions to find the optimal separating hyperplane in a high-dimensional space.

K-Nearest Neighbours (KNN): A non-parametric instance-based classifier.

Neural Networks (Multi-Layer Perceptron): To model complex multi-layered feature interactions.

The evaluation methodology focuses primarily on measuring overall predictive Accuracy as the core success metric to determine which mathematical approach yields the highest detection rate on the unseen test split.

2. Critical Evaluation

• The Main Claims Made by the Author

The primary claim made by the authors is that machine learning models, specifically ensemble techniques and rule-based classifiers, can achieve extremely high accuracy (up to 97%) in detecting phishing websites using the 30 features extracted from the UCI dataset. They claim that their framework offers a highly robust, dynamic solution that can effectively replace or heavily supplement traditional static blacklists. Furthermore, they imply that by evaluating various classification algorithms (such as Random Forest, SVM, and Neural Networks), they have successfully identified the optimal computational approaches for real-time phishing mitigation.

• Whether the Claims are Supported by the Presented Evidence

While the empirical results presented in the paper (the high accuracy percentages) match the outputs of their specific code execution, a deeper look reveals that the evidence does not fully support the claim of a "robust, real-world ready" solution. The high accuracy is achieved within a controlled, academic environment using a heavily pre-processed, static dataset.

- **The Nature of the Data:** The evidence is based on a dataset where features are already perfectly engineered and discretised into -1, 0, and 1. In a real cyber defence environment, extracting these 30 features perfectly from a live, obfuscated webpage without significant latency is highly challenging.
- **The Time Factor and Dataset Age:** Phishing is a highly temporal problem: attackers constantly change their methods. The authors prove that their model works well on a specific historical snapshot of data, but because the dataset age is highly significant, they do not provide evidence that this accuracy holds over time or against evolving evasion techniques.

- **Artificial Class Distribution:** The dataset collection methodology enforces an optimistic, balanced class prevalence (roughly 55% legitimate and 44% phishing). In actual production environments, phishing traffic is a rare anomaly representing less than 1% of daily network volume. The presented evidence fails to prove that the model can handle real-world distribution ratios without triggering a catastrophic number of false alarms.

- **Whether the Evaluation Methodology is Appropriate**

The evaluation methodology applied in the paper has several significant flaws when looked at through a cybersecurity lens:

- **Over-reliance on Accuracy:** The authors heavily focus on Accuracy as their primary metric for success. In cybersecurity, and specifically phishing detection, general accuracy is a misleading metric. We care deeply about the specific types of errors the model makes (False Positives vs. False Negatives).
- **Standard Train-Test Split (80/20) and Data Leakage:** The authors used a random train-test split. For standard tabular data, this is common, but for phishing, it creates a severe risk of optimistic bias and unmitigated data leakage. Because the original methodology failed to clean duplicated or near-duplicated web samples before splitting, identical phishing landing pages appear across both the training and testing sets. This allows the decision trees to simply memorize recurring static rows rather than learning generalized malicious patterns.
- **Lack of Temporal Split:** Because phishing campaigns often use similar URL structures within the same timeframe, a random split masks the model's inability to generalize to completely new, unseen phishing campaigns. A Temporal Split (training on older data, testing on newer data) would have been much more appropriate to evaluate true real-world generalization.

• Possible Weaknesses or Limitations

Several critical weaknesses and limitations can be pointed out in the original work:

- **Outdated Feature Assumptions:** The dataset relies on older indicators of phishing, such as the presence of an @ symbol or specific sub-domain structures. Modern phishing kits use advanced techniques like reverse proxies, hidden HTML elements, canvas rendering, and legitimate cloud service hosting (e.g., AWS, Google Cloud), which completely bypass these 30 traditional features and fail to generalize to modern attacks.
- **Lack of Latency Analysis:** For a phishing detection model to be integrated into an endpoint protection agent or a browser extension, it must evaluate a URL in milliseconds. The authors do not discuss the computational complexity or the inference time of their models (especially the complex Neural Network or Random Forest with many trees).
- **No Discussion on Concept Drift:** In cybersecurity, "Concept Drift" happens when the behaviour of malicious actors' changes over time. The paper completely ignores how the model would degrade when attackers adapt and change their tactics to evade the defined rules.

• Whether the Conclusions are Justified

The authors' conclusion that machine learning is highly effective for phishing detection is justified in theory, but their specific conclusion that their model is ready to solve the practical problem is overblown. Their high success rate is a product of a "clean" dataset.

Replication Insight: As we bridge into our own implementation, our findings show that while we can successfully replicate the high accuracy scores on this specific static dataset, the system suffers from a lack of adaptability. Since our findings and corrected model evaluations show a defined rate of False Negatives (missing phishing sites) and False Positives (blocking safe sites), the authors' conclusion that this is a complete solution is compromised. A high-performing model on paper can become a major

operational burden in a real-time Security Operations Center (SOC) due to alert fatigue from False Positives.

Section 3: Feature Engineering Analysis

In this project, a structured feature engineering and preprocessing pipeline was successfully executed before training our machine learning models. The final architecture utilized twenty-nine specific cybersecurity features extracted from the website data space, down from the initial thirty input features provided in the tabular dataset. Regarding structural transformations, normalization, and scaling techniques, such as logarithmic modifications, Box-Cox transformations, StandardScaler, or MinMaxScaler, all were intentionally omitted from our final pipeline. The mathematical justification for excluding feature scaling is that distance-based models like K-Nearest Neighbors or Support Vector Machines require uniform scaling because features with larger numerical ranges dominate gradient updates. However, all features in this benchmark dataset are already strictly bounded within an identical discrete categorical integer scale of minus one, zero, and one. Because all indicators share the exact same mathematical boundaries, no single feature can dominate the others due to scale discrepancies. Furthermore, our downstream classifiers are tree-based ensembles, which are completely invariant to monotonic feature scaling, meaning that applying standardization would increase computational overhead without providing any mathematical benefit to data quality or model accuracy.

Similarly, we explicitly chose not to apply standard categorical encoding methods like One-Hot Encoding or Target Encoding to the input features. Instead, we preserved the original discrete integer mapping where minus one represents Phishing, zero represents Suspicious, and one represents Legitimate. Because our predictive models are tree-based architectures like Random Forest and XGBoost, their internal mathematical splitters, such as the CART algorithm, branch out by isolating integer boundaries natively. If we had applied One-Hot Encoding to all thirty features, it would have artificially inflated our dimensionality from thirty columns to nearly ninety columns. Spreading the statistical signal across a massive, mostly empty sparse matrix triggers the curse of dimensionality, increases memory footprint, and slows down real-time

model evaluation latency without adding any predictive value or interpretability to the system.

System redundancy represents a major challenge in tabular cybersecurity datasets, and it can be programmatically spotted using a Spearman rank correlation matrix during the Exploratory Data Analysis phase. Spearman correlation is the correct mathematical tool here because our data consists of discrete ordinal integers rather than continuous variables, allowing us to find monotonic relationships. By calculating this matrix, we programmatically exposed severe structural redundancy, discovering that the popUpWidnow feature and the Favicon feature shared an extreme rank correlation of zero point ninety-four. To tackle this issue and enforce structural parsimony, we executed manual feature selection by completely dropping the popUpWidnow feature from both the training and testing sets. This transformation directly affected model performance in a highly positive way. It successfully reduced the complexity of the feature space, prevented the decision trees from building overlapping, unstable boundaries on collinear variables, and kept the pipeline lightweight enough to be deployed inside resource-constrained production environments like a real-time web browser extension.

The entire feature engineering process proved to be highly meaningful, balancing solid mathematical intuition with critical cybersecurity domain logic. Mathematically, dropping a redundant collinear feature ensures that our bagging and boosting tree nodes choose independent, high-variance vectors at each split, which directly protects the system against overfitting and stabilizes feature importance values. From a cybersecurity perspective, this process is meaningful because a real-time phishing filter must operate with minimal computational latency. Eliminating redundant checks means the system requires fewer parsing operations on the client side, allowing the browser extension to flag malicious websites instantly before the page completely loads.

Finally, while we are restricted to a pre-discretised tabular benchmark dataset in this notebook, we can identify additional features that could potentially improve real-world detection performance. In a live production environment, we could inject a dynamically computed feature called Visual_Similarity_Score. This feature would use a deep

computer vision model to compare the graphical user interface of the active webpage against a known database of trusted brand logos, immediately flagging visual clones. Additionally, adding a dynamic network-layer feature like `Domain_Age_Days` by querying WHOIS servers in the background would provide vital contextual evidence, as new phishing domains are highly ephemeral and rarely exist for more than a few days, giving the model an extra layer of structural security against zero-day exploits.

Section 4: Reproducibility Analysis

Ensuring strict reproducibility is a fundamental requirement of any modern data science framework, especially within cybersecurity where defensive systems must deliver predictable, validated results. Based on our practical implementation, the entire project notebook can be executed successfully from start to finish without generating any software errors, memory overflows, or unexpected execution bottlenecks. The computational pipeline runs dynamically within a standard interactive Python environment, producing consistent diagnostic metrics and visual confusion matrix charts across consecutive runs.

Every single required dataset file and software dependency needed to reproduce these experimental results is fully available and clearly defined within the repository ecosystem. The primary source file, `dataset.csv`, is packaged locally in the root directory alongside the development notebook, removing any dangerous reliance on external cloud links or broken web download URLs that could compromise future reproducibility. Structurally, the runtime environment relies exclusively on standard open-source data science libraries, specifically `pandas` and `numpy` for matrix manipulation, `matplotlib` for visual rendering, and `scikit-learn` and `xgboost` for mathematical modeling. Because all components use stable, well-documented package distributions, any engineer can duplicate the exact runtime environment within moments.

A critical inspection of the implementation confirms that no hidden preprocessing steps, hardcoded manual data adjustments, or silent feature filtering operations exist within the framework. Every single stage of data modification, from dropping the

uninformative index column during initial setup to dropping the redundant popUpWidnow variable during feature selection, is explicitly written out as a clear, modular function inside the visible cell workflow. By avoiding external script files or manual spreadsheet editing, we guarantee that the transition from raw tabular text data to optimized feature vectors is fully transparent, auditable, and scientifically verifiable.

The overall reproducibility of this work is exceptionally high and scientifically valid. To guarantee absolute consistency across different machines, we implemented a strict stratified train-test split that keeps class prevalence uniform, combined with a fixed random seed of forty-two applied directly to all data splitters and ensemble classifier initializations. By binding the internal randomness of the bagging and boosting decision branches, we removed all statistical variance from the pipeline. As a result, anyone executing this project on a different machine will obtain the exact same ninety-seven point fifty-one percent accuracy for the Random Forest model and ninety-seven point twenty-four percent accuracy for XGBoost, demonstrating a flawless benchmark for open-science replication.

Section 5: Experimental Results

To evaluate the feasibility and performance of tree-based machine learning for automated cyber defense, we structured a complete empirical experiment inside our integrated development notebook. The core experiment involved building a baseline supervised pipeline capable of taking static website characteristics and outputting a binary prediction to separate safe browsing environments from malicious credential-harvesting destinations. The experimental environment was designed around a clean, modular python framework utilizing a static eighty-twenty train-test split that applied stratification to perfectly preserve class proportions across both data segments. To ensure that our experimental results are scientifically valid and completely repeatable by any developer, we introduced a fixed random seed of forty-two across all data partitions and model initializations.

During the development lifecycle, we introduced a major architectural modification to the baseline framework to address feature redundancy and optimize system speed. Our

early exploratory data analysis discovered an extreme Spearman rank correlation of zero point ninety-four between the popUpWidnow feature and the Favicon feature. To prevent the downstream decision trees from building unstable boundaries on highly collinear variables, we applied feature selection to drop the popUpWidnow indicator entirely, optimizing the feature space from thirty down to twenty-nine descriptive inputs. We then trained two highly powerful and distinct ensemble machine learning models on this refined data space: a Random Forest Classifier representing a bagging framework, and an XGBoost Classifier representing a sequential gradient boosting framework. Because the XGBoost objective function inherently requires non-negative target indices starting at zero, we seamlessly integrated a LabelEncoder into the boosting channel to safely map the target labels from minus one and one to zero and one.

To evaluate the operational success of our systems in a manner meaningful to the phishing domain, we avoided blind reporting and selected a comprehensive suite of classification metrics. We utilized overall Accuracy to measure general correctness, Precision to evaluate the exact purity of our security alerts, and Recall to measure our true detection rate or sensitivity. Furthermore, we computed the F1-Score as the harmonic mean to assess the balance between false alarms and missed threats, alongside the Matthews Correlation Coefficient to ensure our findings remain invariant to any real-world class distribution imbalances. Finally, we extracted raw operational counts from the Confusion Matrix and measured the Area Under the Receiver Operating Characteristic curve to verify the overall mathematical stability of our probabilistic decision boundaries.

The obtained experimental results demonstrate phenomenal predictive success for both ensemble systems, with the Random Forest model emerging as the superior framework for cybersecurity deployment. In our testing phase, the Random Forest model achieved an overall accuracy of ninety-seven point fifty-one percent, compared to the ninety-seven point twenty-four percent recorded by XGBoost. Looking at the threat-mitigation metrics, Random Forest delivered a precision of ninety-seven point nineteen percent and a recall of ninety-eight point thirty-eight percent, resulting in a balanced F1-Score of ninety-seven point seventy-eight percent and a Matthews Correlation Coefficient of zero point ninety-four ninety-six. In contrast, XGBoost

achieved a precision of ninety-six point eighty-eight percent, a recall of ninety-eight point twenty-one percent, an F1-Score of ninety-seven point fifty-four percent, and a Matthews Correlation Coefficient of zero point ninety-four forty-one. The high ROC-AUC scores of zero point ninety-nine seventy-seven for Random Forest and zero point ninety-nine seventy-two for XGBoost confirm extreme boundary stability. Most importantly, looking at the raw confusion matrix counts, Random Forest minimized false positives down to just twenty while generating thirty-five false negatives, outperforming XGBoost which allowed twenty-two false positives and generated thirty-nine false negatives. These empirical results prove that while both models are highly capable, the bagging architecture provides better perimeter protection against phishing attacks.

Section 6: Conclusions and Future Work

The final phase of this project allows us to consolidate our empirical evidence, reflect on systemic insights, and outline strategic directions for future development. The key findings of this study prove that tree-based machine learning ensembles are highly capable of detecting phishing infrastructure with exceptional accuracy using static layout features. Through rigorous testing, the Random Forest architecture emerged as the superior system, achieving a top overall accuracy of ninety-seven point fifty-one percent, compared to the ninety-seven point twenty-four percent recorded by XGBoost. More importantly, from a threat mitigation standpoint, Random Forest demonstrated a superior recall of ninety-eight point thirty-eight percent, successfully catching more credential-harvesting threats and minimizing operational false positives down to just twenty instances. These results confirm that utilizing automated ensemble voting structures provides a reliable perimeter defence framework against phishing vectors.

The most critical lessons learned throughout this data science lifecycle involve the hidden danger of system redundancy and the operational impact of distinct error types. Our exploratory analysis taught us that tabular datasets often contain heavy multicollinearity, as seen by the extreme zero point ninety-four rank correlation between popUpWidnow and Favicon. Learning how to identify and tackle this redundancy by dropping the duplicate feature manually taught us that streamlining the feature space

directly improves boundary stability and helps prevent model overfitting. Furthermore, evaluating the practical impact of errors taught us that false negatives and false positives carry completely unequal operational costs. A false negative compromises enterprise security by letting a live threat pass, while a false positive disrupts business continuity by blocking safe websites, meaning that final models must be tuned with realistic deployment environments in mind rather than focusing blindly on generic accuracy scores.

Evaluating the strengths and weaknesses of our final proposed solution reveals a clear architectural trade-off. The primary strength of our Random Forest approach is its high operational purity, low classification latency, and complete invariance to feature scaling, making it incredibly lightweight and suitable for client-side deployment inside a web browser extension. However, a distinct weakness of the system is its complete reliance on static snapshot features and highly curated benchmark data. Because the model operates under the optimistic assumption that phishing traffic distributions remain balanced, it is highly vulnerable to data drift. In the real world, phishing campaigns change rapidly, meaning that our static tree splits will inevitably decay over time as attackers adjust their techniques to bypass established rules.

To overcome these limitations, we can propose clear suggestions for future improvements to enhance long-term system resilience. First, the framework should be updated to support dynamic feature engineering by integrating real-time WHOIS queries to calculate domain age, alongside a visual similarity analyzer that uses computer vision to detect graphical brand clones. Second, the single static classifier should be converted into a continuous learning pipeline that automatically updates its decision trees using live threat intelligence feeds. Finally, to deploy this project safely on a similar real-world problem without risking business disruption, the classification model should be paired with an automated corporate whitelist and configured with a slight mathematical bias toward higher recall, guaranteeing maximum security while maintaining a smooth, reliable user experience.

